

Introduction to Java

The Java Environment (AWS, n.d.a.)

Java is a free-to-use and highly dynamic high-level programming language for coding programs such as web applications, mobile applications, enterprise software, big data applications, and server-side technologies. It was created by Sun Microsystems in 1995 as a multi-platform, object-oriented, and network-centric language that can be used as a platform in itself.

The common uses of Java include game development, cloud computing, big data, artificial intelligence (AI), and Internet of Things.

Java Runtime Environment (JRE)

It is the software that Java programs require to run correctly. It provides the *class libraries* and other resources that any Java program needs.

The advantages of using Java include the following:

- **Active Community Support:** Given its popularity and longevity, Java has many active users and a community that can support developers.
- **High-Quality Learning Resources:** Learning resources such as detailed documentation, comprehensive books, and courses are available to new programmers for additional support.
- **Inbuilt Functions and Libraries:** Developers do not need to write every new function from scratch, as Java provides a rich ecosystem of built-in functions and libraries to develop various applications.
- **Security:** The Java security levels and restrictions are highly configurable. An untrusted Java code can be downloaded and run in a secure environment in which it cannot do any damage. The untrusted code also cannot damage the host system with a virus, nor can it read or write files from the hard drive.
- **Platform Independent:** Java code can be written once and run on any underlying platform like Windows, iOS, Android, and Linux without rewriting. This feature is called the “**Write Once, Run Anywhere**” (WORA) Philosophy which became a highly sought feature by developers.

To use Java, a developer must understand two (2) things: Java Virtual Machine and Java Language and APIs.

Java Virtual Machine (JVM)

It is a virtual machine that runs Java applications as a run-time engine. Natural programming languages fall into two (2) broad categories: compilers and interpreters.

- **Compilers:** The program is written in natural English-like syntax with compilers, and the language then translates the entire code into machine language. It converts the source code into a binary program of bytecode.
- **Interpreters:** This checks the bytecode and communicates with the operating system, executing the bytecode instructions line by line.

Java was the first programming language to combine both using a **Java Virtual Machine (JVM)**. A JVM is an environment that translates Java bytecode, an intermediate-level language, into machine language and executes it.

The **programming statements** or **source code** are stored on a disk with a *.java* file extension. The **Java compiler** converts the source code into a binary program of bytecode. The **Java interpreter** will then check the bytecode and communicate with the operating system, executing the bytecode instructions line by line within the Java

Virtual Machine, named *java.exe*.

Java APIs

APIs (Application Programming Interface) are important software components bundled with the Java Platform. **Java APIs** are pre-written Java programs that plug and play existing functionality into a code, such as getting the time and date, performing mathematical operations, or manipulating text. The Java language has only about 50 keywords, but the Java API has several thousand classes, with thousands of methods that can be used in any program.

Java Skeleton (Soma, T. & Streib, J., 2023)

The best way to understand a programming language is by dissecting and examining its sample program.

Here is a simple example of a Java program that prints or displays “First Java Application”:

```

1 class Skeleton {
2     public static void main (String[] args){
3         System.out.println("First Java Application"); //for display
4     }
5 }
```

Line 1 begins with the *reserved word* or *keyword* **class**. A **class** is the basic unit of a Java program. It also defines a group of *objects* that includes *data members*, which are places to store data, and *methods* which are places to put the program logic. Think of a class as a blueprint for a house and the house itself as the object. On the same line, the word *Skeleton* is the class name provided by the programmer.

A class name is a type of an identifier. An **identifier** is the name given to a program component, such as a class, object, or variable. These are the requirements for naming an identifier:

- It must begin with a letter of the English alphabet, an underscore (_), or a dollar sign (\$), but **not** with a digit.
- It can have any combination of letters, digits, underscores, or dollar signs, but **not** with white spaces.
- It is case-sensitive such as the identifier **NUMBER** is not the same as the identifier **number** or **Number**.
- It cannot be one of the following values: **true**, **false**, or **null**. These are not keywords; they are primitive values but reserved and cannot be used.
- It cannot be a reserved word.

Reserved Words

Reserved words or **keywords** are special words significant to the Java compiler. Aside from class names, these cannot be used as *variables*, a name used to refer to data stored in the memory. Here is a list of Java keywords:

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

Still, on line 1, **braces** are used to identify blocks of code and data. It requires matching opening ({) and closing braces (}). Thus, the whole definition of the class `Skeleton` should be between the first opening brace and the last closing brace on line 5.

The `Skeleton` class has one (1) *method* definition on line 2. The *method* is intended to improve the readability of the program. The first three words (`public static void`) in line 2 are reserved words.

The **public** keyword is a type of access or visibility modifier. This keyword is used to define a `main()` method so that the interpreter can execute the program.

The **static** keyword tells that this is a *class* method. The `main` method is always declared `static` so that it can be executed without creating an object of the class.

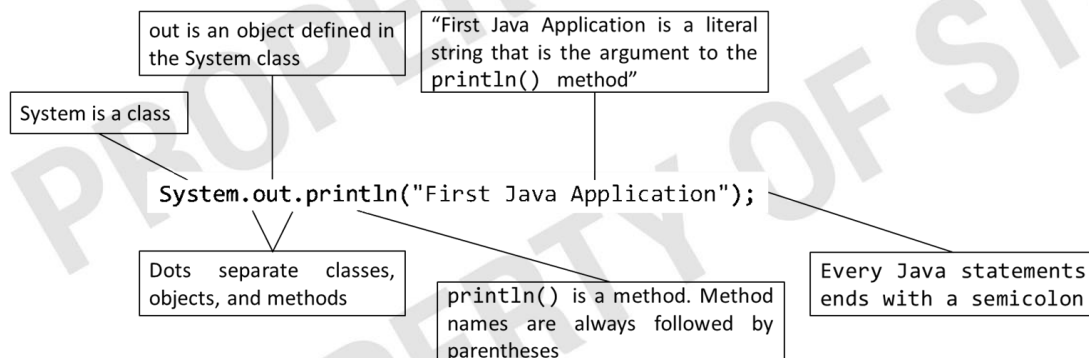
The **void** keyword means that the `main` is a non-value-returning method, which means the `main` is not expected to return any value.

The word **main** after the three (3) keywords is the name of the method. When executing a Java program, the execution must always begin with the `main` method. All Java applications must include a class containing one (1) `main` method.

Inside the parentheses after the method name, **parameters** are listed along with their types to allow the method to receive values. The `main` method has a parameter called `args`, an array of type `String`, and the *square brackets* `[]` indicate `args` is an array. An **array** is a collection of similar data types, while a string is a sequence of characters containing letters, numbers, and symbols.

The definition of the `main` method starts with an opening brace (end of line 2) and ends with a closing brace (line 4).

This sample Java program contains one (1) programming statement or command in line 3:



A **literal string** is a series of characters that appear exactly as written and is always enclosed by double quotation marks (" , "). **Arguments** are information passed to a method so it can perform its task. A **Java statement** such as the one above is an executable instruction that commands the compiler what to do.

A comment can also be found at the end of line 3: `//for display`. **Comments** provide additional information to make programs easier for programmers and other programmers to understand. Comments will not be compiled and executed when the program is run. A comment line can start with `//` symbols and continue to the end of the line. This type of comment line is used for a single-line comment. Comments can also be placed between `/*` and `*/` symbols if the comments run over multiple lines.

Saving, Compiling, and Executing a Program (Agarwal, S. & Bansal H, 2023)

Java Development Kit (JDK) includes complete JRE tools for developing, debugging, and monitoring Java applications. JDK must be installed on the computer to write, compile, and execute Java programs.

Here is a sample program to be saved, compiled, and executed using the command prompt (CMD):

Step 1: Write the following code in a text editor such as Notepad (or any text editor.)

```
public class Demo {  
    //program begins from here starts with  
    public static void main (String a []) {  
        System.out.println("First program in Java");  
    }  
}
```

Step 2: If using a Notepad as an editor to write a program, the file must be saved with double quotes (" "). Otherwise, it automatically adds a .txt extension. The sample program above should be saved as "Demo.java".

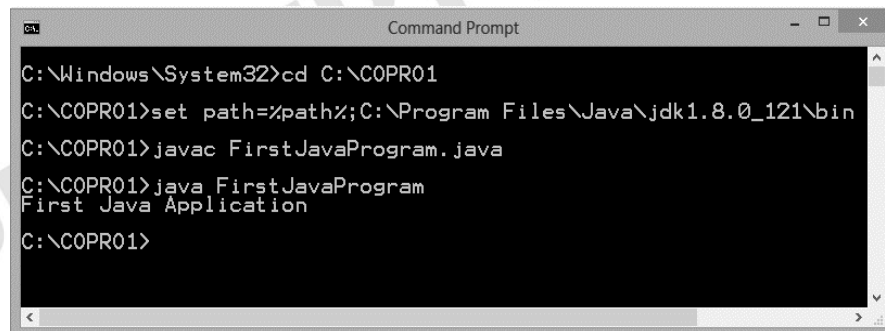
Step 3: Open the DOS window (cmd), then change the directory to the directory where the Java file was saved.

Step 4: Set the path of JDK using the path command. This setting tells the operating system where to find the Java compiler and the classes. Setting the path of JDK is defined as **set path=%path%;[JDK bin directory]**

Step 5: Compile the Java program using the javac command: **javac [name of the Java file].java**

Step 6: Execute the compiled Java program using the java command (interpreter): **java [name of the Java file]**

Output:



```
Command Prompt  
C:\Windows\System32>cd C:\C0PR01  
C:\C0PR01>set path=%path%;C:\Program Files\Java\jdk1.8.0_121\bin  
C:\C0PR01>javac FirstJavaProgram.java  
C:\C0PR01>java FirstJavaProgram  
First Java Application  
C:\C0PR01>
```

Figure 1. Output. Retrieved from Agarwal, S. & Bansal H. (2023). *Java in depth*. BPB Publications.

References:

Agarwal, S. & Bansal H. (2023). *Java in depth*. BPB Publications.

AWS (n.d.a). *What is Java?* [Web Article]. Retrieved on February 7, 2024, from <https://aws.amazon.com/what-is/java/>

Soma, T. & Streib, J. (2023). *Guide to Java: A concise introduction to programming*. Springer.